

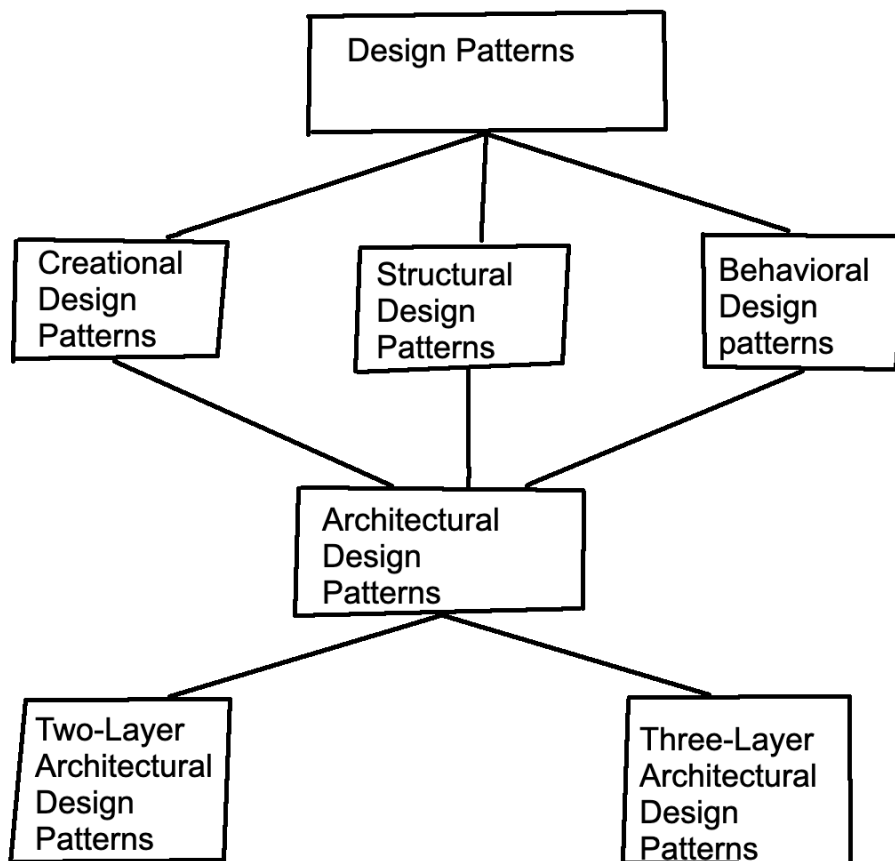
Вступление

Раздел “Классификация паттернов проектирования” необходим для новичков.

Раздел “Параметры для сравнения” необходим для использования в разделе “Сравнение архитектурных паттернов проектирования”.

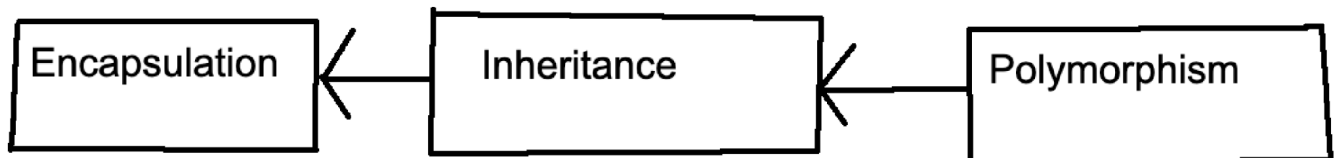
Раздел “Сравнение архитектурных паттернов проектирования” необходим, потому что это основная часть статьи.

Классификация паттернов проектирования



Архитектурные паттерны проектирования создаются благодаря порождающим, структурным, поведенческим паттернам проектирования. Пример: Существует MVVM (Model, View, ViewModel), который является трехслойным архитектурным паттерном проектирования. Для обновления UI (User Interface) в файлах View и ViewModel, используется Наблюдатель или Издатель-Подписчик, которые являются поведенческими паттернами проектирования.

Архитектурные паттерны проектирования были созданы, чтобы разделить логику (Model) и представления (View) используя ООР (Object-Oriented Programming). ООР делиться на три основных принципа (Абстракцию затрагивать не будем), которые последовательно зависят, кроме инкапсуляции. Инкапсуляция - это сокрытие исходного кода в переменных, методах, и классах. Наследование - это расширение класса на потомков, где потомок может получить доступ к переменным, и методам предка. Полиморфизм - это замена реализации одного потомка класса на другого.



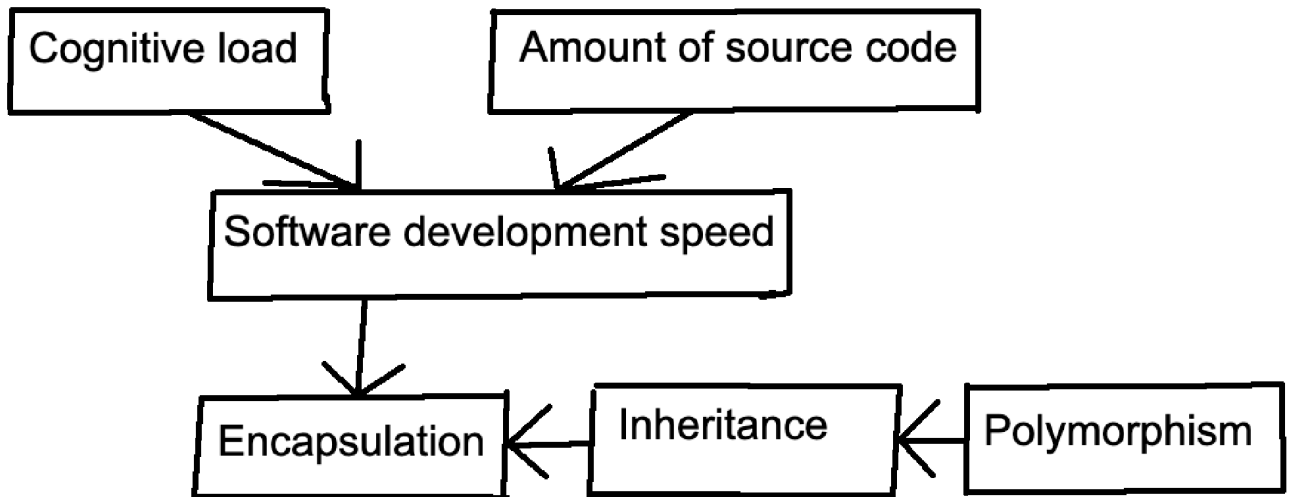
- An arrow pointing at a square represents a dependency of the square that directs that arrow.

Полиморфизм зависит от Наследования, а Наследования зависит от Инкапсуляции.

Параметры для сравнения

Architectural Design Patterns	Software development speed	
	Cognitive load	Amount of source code

Бизнесмены и инвесторы хотят создавать и обновлять ПО (Программного Обеспечения) как можно быстрее, потому что существует конкуренция, и поэтому параметр скорость разработки ПО является ключевым параметром для определения лучшего архитектурного паттерна проектирования.



- An arrow pointing at a square represents a dependency of the square that directs that arrow.

Параметры когнитивная нагрузка, и количество исходного кода зависят от параметра скорость разработки ПО, а параметр скорость разработки ПО зависит от инкапсуляции. Вообще, все зависит от инкапсуляции.

Инкапсуляция

Инкапсуляция - это сокрытие исходного кода в переменных, методах, и классах. Пример: Без инкапсуляции, это когда в исходном коде написан один оператор if. А с инкапсуляцией, это когда в исходном коде написан метод, и в теле этого метода, написан один оператор if.

Когнитивная нагрузка

Когнитивная нагрузка - это термин в когнитивной психологии (Когнитивная психология изучает как работает память человека), который утверждает, что рабочая память (Или как известно кратковременная память) имеет ограничения.

Цитирую от [Cognitive load - Wikipedia](#): "Миллер, возможно, первым предположил, что объем рабочей памяти человека имеет внутренние ограничения. Его экспериментальные результаты показали, что люди, как правило, способны удерживать в кратковременной памяти только семь плюс-минус две единицы информации (От 5 до 9 единиц информации)."

Я называю "единицы информации" фрагментами времени. Значение одного фрагмента времени равняется до одной миллисекунды, секунды, минуты, часа времени. Значение фрагмента времени зависит от сложности условия в операторе if, и инкапсуляции. То есть у одного фрагмента времени, может быть значение равным 200 миллисекундам, или 30 минутам.

Как тратятся фрагменты времени ? На запоминание условия в операторе if, или на запоминание имени переменной, или метода, которая описывает условия, и находится в операторе if, а если

имя переменной, или метода не дает полного описания условия, то еще тратим фрагмент времени на запоминание условия в теле переменной, или метода.

Сколько тратиться фрагментов времени ? Один фрагмент времени равен одному оператору if, или условию в теле переменной, или метода. Пример:

Without encapsulation (Exception: method) void main() {}	With encapsulation
<pre>void main() { if(2 != 5 && 5 == 5 && 3 > 1) { +1 (2.0 second) print("firstCondition"); return; } if(4 != 4 && 1 > 3) { +1 (1.2 second) print("secondCondition"); return; } }</pre>	<pre>void main() { if(firstCondition()) { +1 (1.0 second) print("firstCondition"); return; } if(secondCondition()) { +1 (600 milliseconds) print("secondCondition"); return; } } bool firstCondition() { return 2 != 5 && 5 == 5 && 3 > 1; +1 (2.0 second) } bool secondCondition() { return 4 != 4 && 1 > 3; +1 (1.2 second) }</pre>
Fragment of time: 2	Fragment of time: 4

Инкапсуляция повышает когнитивную нагрузку, если имя переменной, или метода не дает полного описания условия.

Количество исходного кода

Количество исходного кода зависит от инкапсуляции. Пример: Есть метод под названием "getHttpStatusMessage(statusCode)", целью которого получить HTTP статус код, и вернуть сообщение пользователю о статусе запроса (Это если статус код не равен 200, то вызывается этот метод). Этот метод можно использовать в любом HTTP-запросе, и вместо копирования и вставки 150-200 строк исходного кода, можно скопировать только названия метода.

Инкапсуляция уменьшает количество исходного кода, потому что переменные, классы, и методы можно переиспользовать в локальных переменных, методах, и классах, то есть копируя только названия метода.

Сравнение архитектурных паттернов проектирования

Architectural Design Patterns	Software development speed	
	Cognitive load	Amount of source code
Two-layer		
Three-layer		

- Evaluation for each parameter is conducted based on priority, ranging from the maximum to the minimum value.

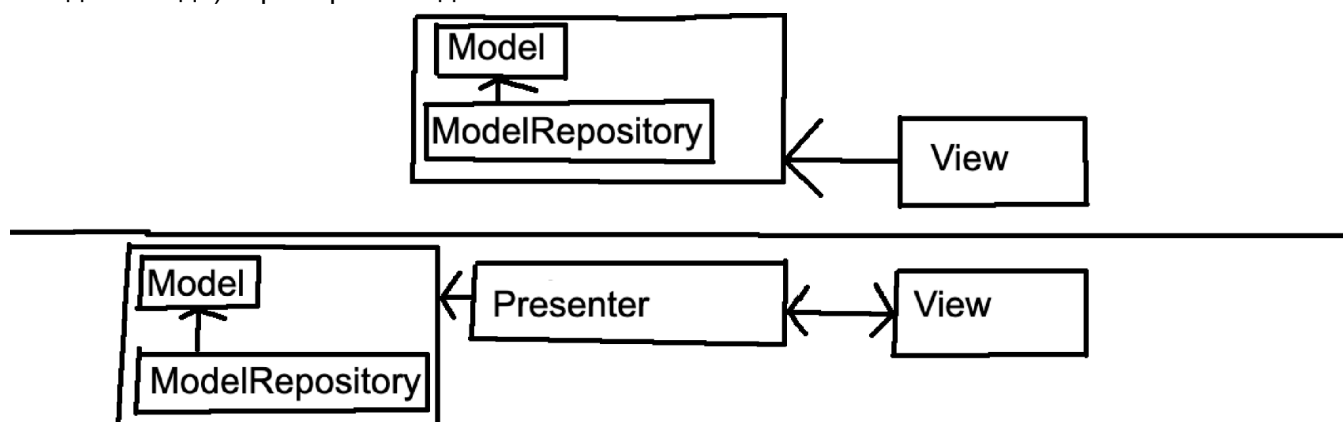
- The formula for the maximum value in this context:

$\text{number_of_architectural_design_patterns} - 1 = 2 - 1 = 1$

На изображении показано, что по каждому параметру когнитивная нагрузка, и количество исходного кода в архитектурных паттернах проектирования, значения должны быть уникальны, потому что необходимо определить победителя, и проигравшего, а если значения будут равны в каждом параметре, то победителя и проигравшего по параметру когнитивная нагрузка, и количество исходного кода не будет.

Сравнение по параметрам, будет описано примерно так:

- 1) Вывод по каждому параметру исходя из раздела "Параметры для сравнения".
- 2) Изображение, где показаны архитектурные паттерны проектирования, и подсчеты по фрагментам времени (Когнитивная нагрузка), и количеству строк исходного кода (Количество исходного кода). Пример без подсчетов:



- An arrow pointing at a square represents a dependency of the square that directs that arrow.

Model - это данные, и может манипулировать с данными.

ModelRepository - это источник данных, который предоставляет данные для Model, то есть сама Model не может обратиться к БД (База Данных) или к сети, чтобы получить эти данные. Model и ModelRepository, это реализация структурного паттерна проектирования Репозиторий (Или как еще называют, это подход DDD (Domain-driven design)).

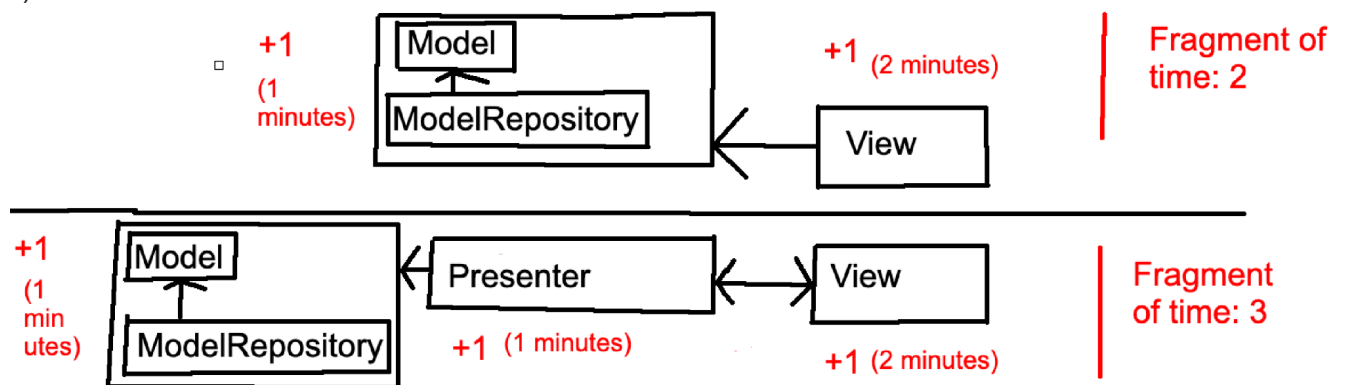
Presenter - это посредник между Model и View, целью которого инкапсулировать классы Model и View, и с наследованием и полиморфизмом классов Model и View, можно будет переиспользовать методы Presenter в других View, что в итоге снизит количество исходного кода.

View - это представление (Или как еще называют, User Interface), которое использует Model или Presenter, чтобы показать данные пользователю.

Сравнение по параметру “Когнитивная нагрузка”

1) В разделе “Параметры для сравнения” по разделу “Инкапсуляция”, и по разделу “Когнитивная нагрузка” цитирую: “Инкапсуляция повышает когнитивную нагрузку, если имя переменной, или метода не дает полного описания условия.”. Вывод: Чем меньше слоев в архитектурном паттерне проектирования, тем ниже когнитивная нагрузка, при условии, если имя переменной, или метода не дает полного описания условия.

2)

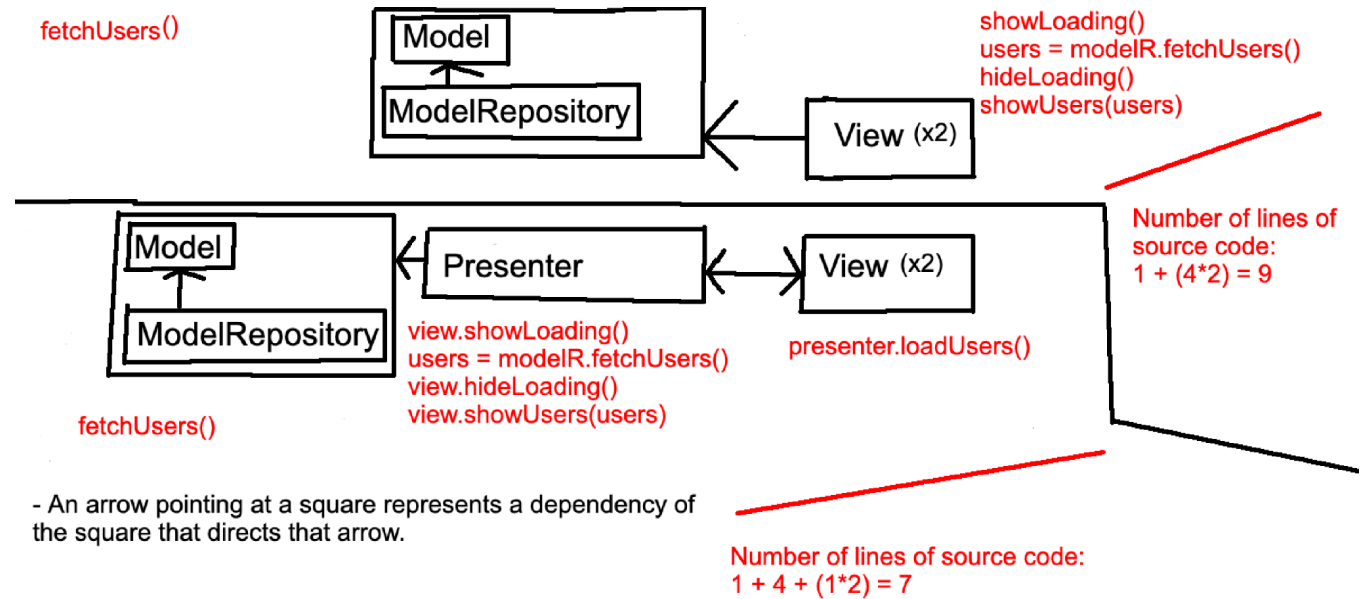


- An arrow pointing at a square represents a dependency of the square that directs that arrow.

Сравнение по параметру “Количество исходного кода”

1) В разделе “Параметры для сравнения” по разделу “Инкапсуляция”, и по разделу “Количество исходного кода” цитирую: “Инкапсуляция уменьшает количество исходного кода, потому что переменные, классы, и методы можно переиспользовать в локальных переменных, методах, и классах, то есть копируя только названия метода.”. Вывод: Чем больше слоев в архитектурном паттерне проектирования, тем меньше количество исходного кода.

2)



Заключение

Architectural Design Patterns	Software development speed	
	Cognitive load	Amount of source code
Two-layer	1	0
Three-layer	0	1

- Evaluation for each parameter is conducted based on priority, ranging from the maximum to the minimum value.

- The formula for the maximum value in this context:

$$\text{number_of_architectural_design_patterns} - 1 = 2 - 1 = 1$$

Если программист быстрее запоминает, то лучше использовать трехслойный архитектурный паттерн проектирования, а если программист быстрее копирует строки исходного кода, то лучше использовать двухслойный архитектурный паттерн проектирования.

Автор статьи: [sl8s](#) (Anton Pichka).